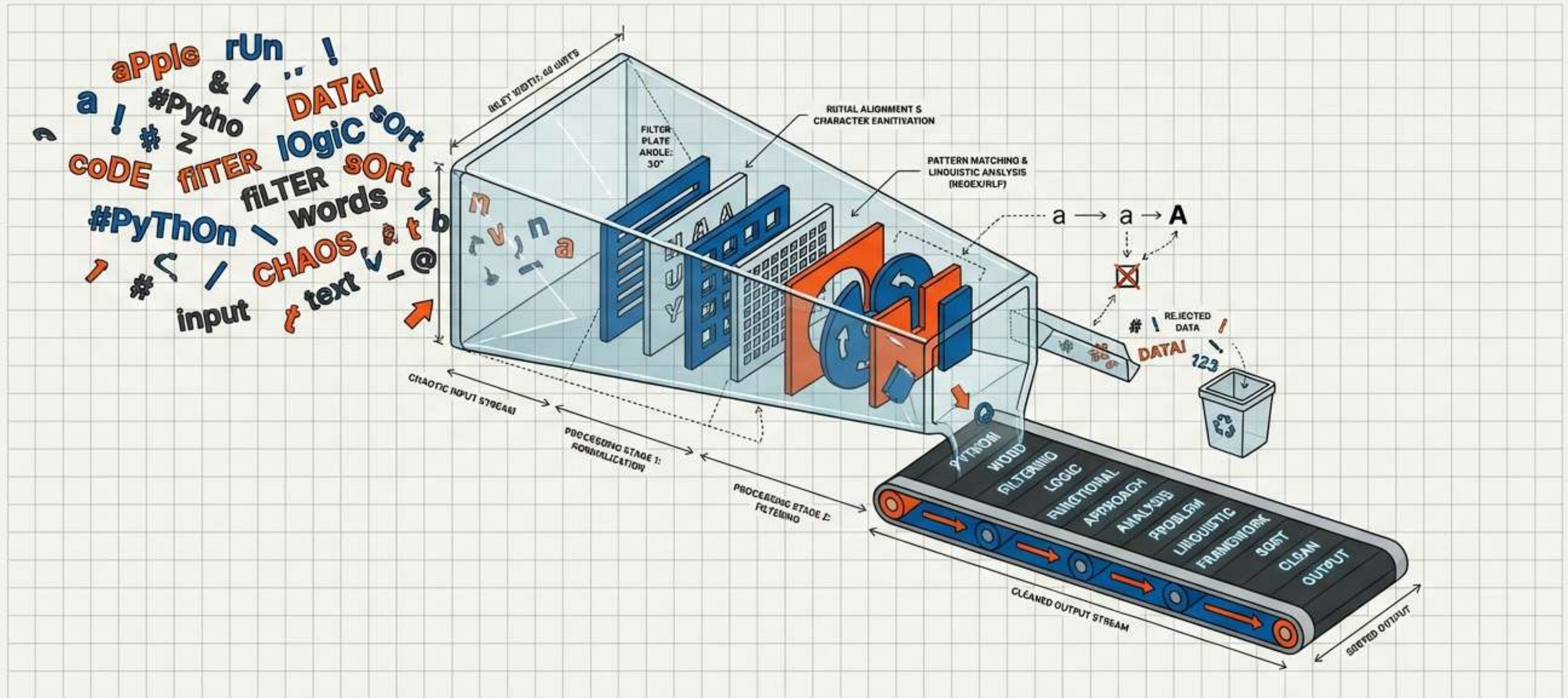


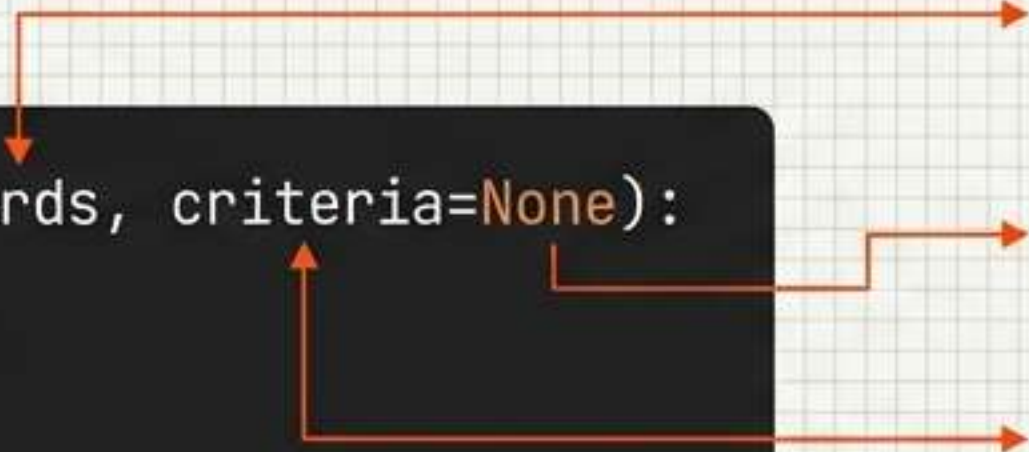
Python Word Filtering Logic: A Functional Approach

Analysis of OPPE 1 (Set 3) Problem 6: The Linguistic Logic Filter Framework



The Problem Statement: Defining the Interface

```
def get_words_by_criteria(words, criteria=None):  
    """  
    Core Function Signature  
    """  
    pass
```



Helvetica Now Display

Input: List[str]. The raw data source.

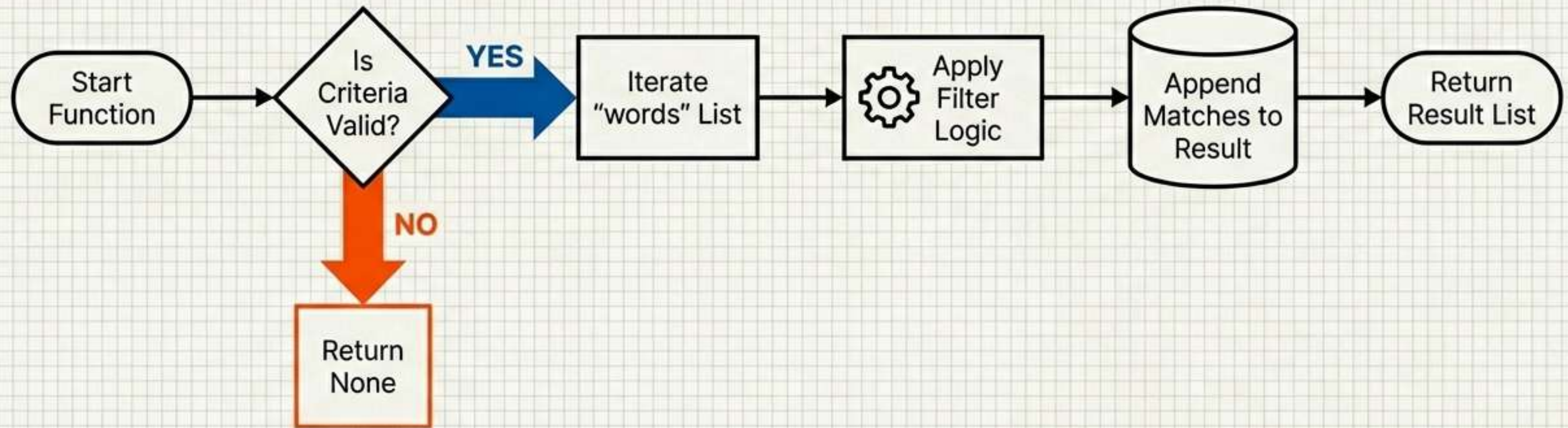
Filter Key: String. The logic selector.

Default: Handles missing arguments gracefully.

Constraints:

Internal logic must be case-insensitive for analysis, but the output must strictly preserve the original casing of the strings.

Logical Architecture & Flow Control



The architecture prioritizes validation. Invalid inputs function as an immediate circuit breaker.

Criterion A: The 'Continuous' Logic

Suffix detection with case-insensitivity.

Code Logic

```
# Check if word ends with "ing" ignoring case
if word.lower().endswith("ing"):
    result.append(word)
```

Valid Matches:

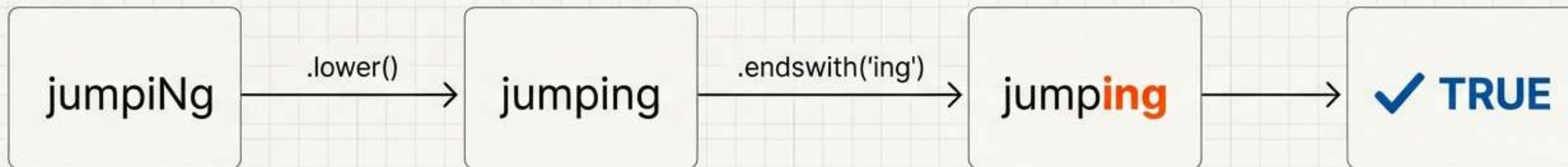
running

jumpiNg

SPINNING

Process Flow: Case-Insensitive Suffix Check

INPUT



Criterion B: The 'Vowel_Rich' Logic

Threshold counting using generator expressions.

Sum(Vowels) > 5

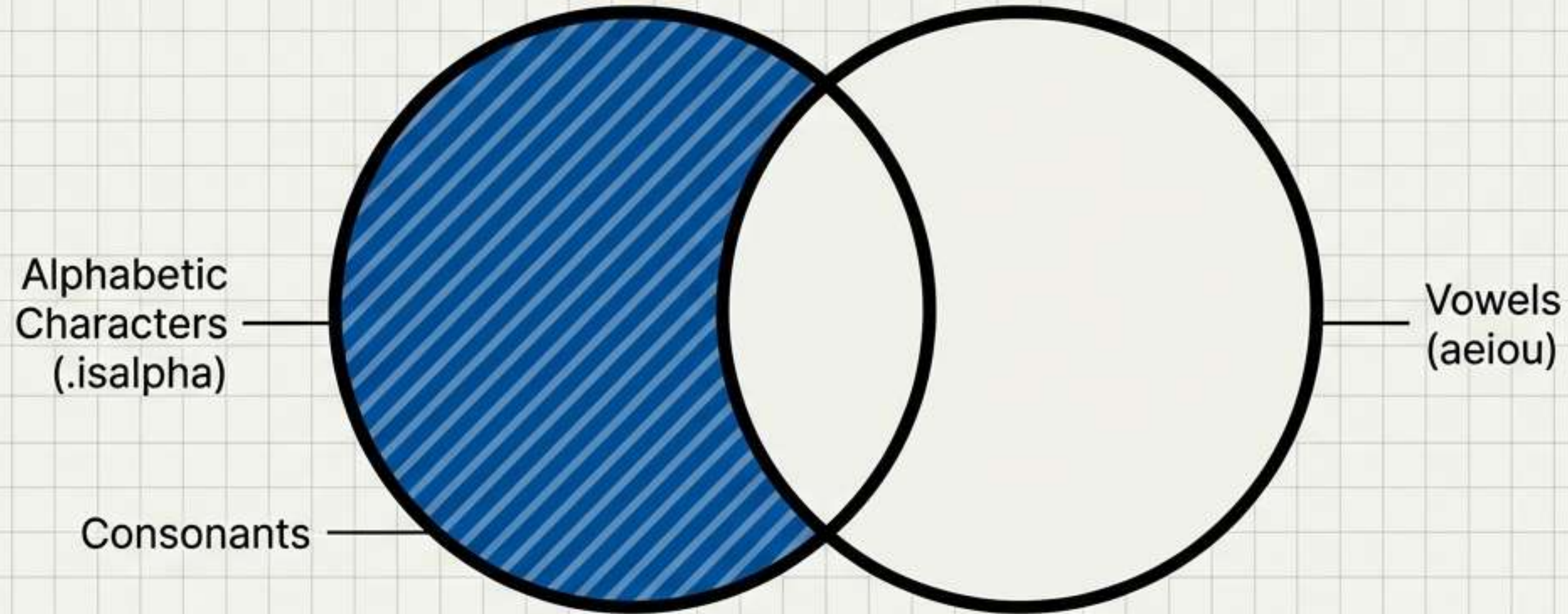
```
# Generator expression for memory efficiency  
vowel_count = sum(1 for c in word.lower() if c in "aeiou")
```

a e i o b c d i o e
1, 2, 3, 4, 5, 6, 7
7 > 5 is TRUE

Why Generator Expressions? They evaluate lazily, avoiding the creation of a temporary list in memory.

Criterion C: The 'Consonant_Rich' Logic

Exclusionary filtering logic using Set Theory.



```
c.isalpha() and c not in "aeiou"
```



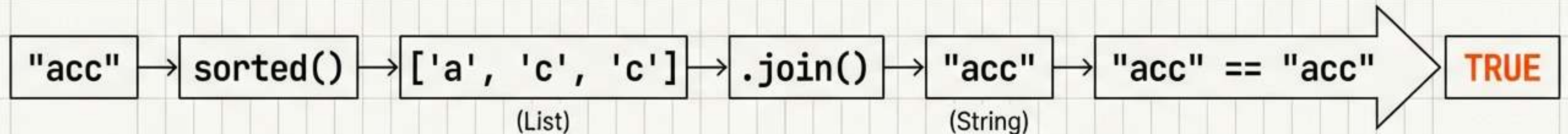
Trap: Symbols and numbers are NOT consonants. Using `not in 'aeiou'` alone is insufficient; we must first verify `.isalpha()`.

SPINNING -> S, P, N, N, N, G (6 Consonants) -> Match.

Criterion D: The 'Sorted' Logic

Lexicographical order validation.

```
word == "".join(sorted(word))
```

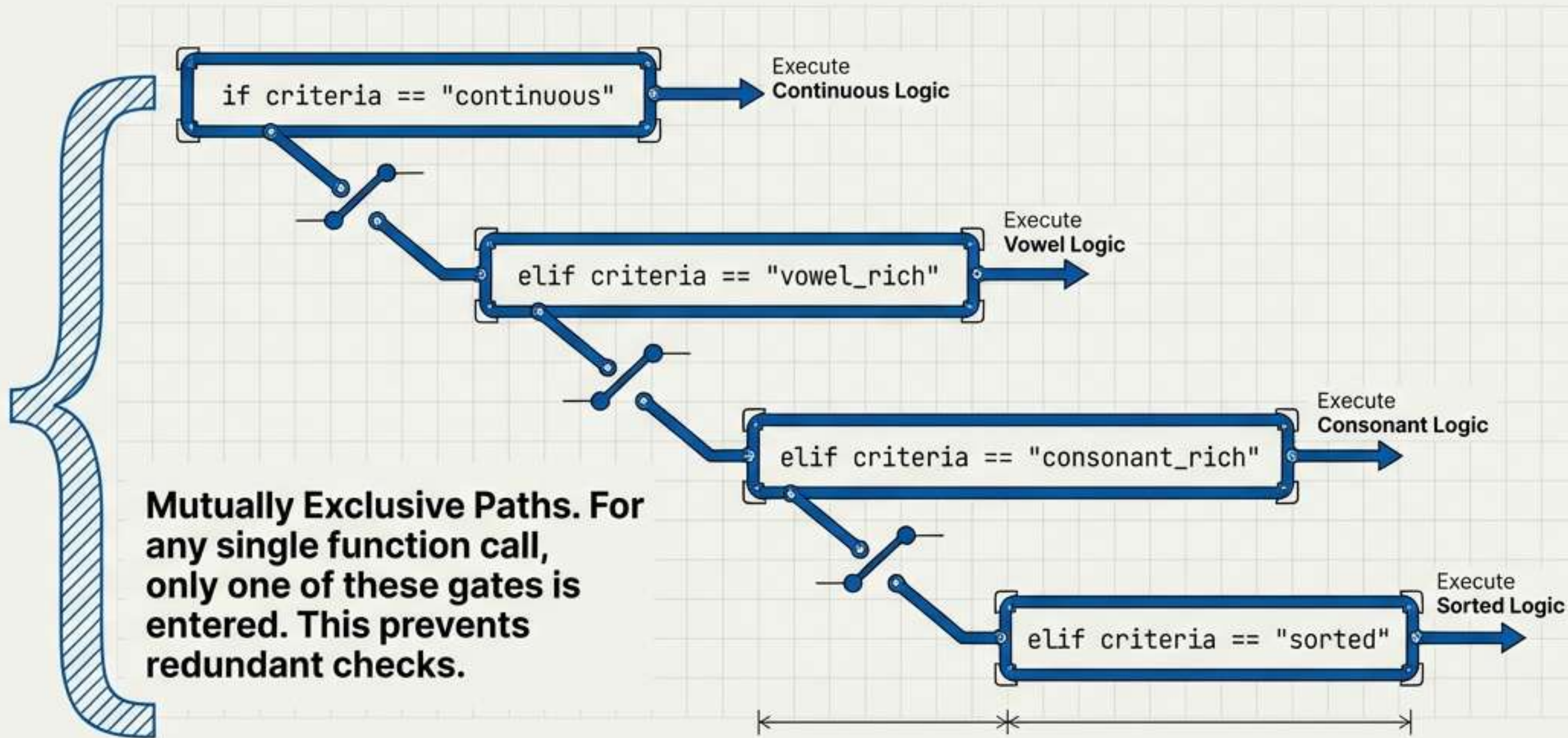


Designer Note

Important: Python sorting is case-sensitive by default.
Upper case (ASCII 65-90) precedes Lower case (ASCII 97-122).
Standard implementation assumes strict ASCII sorting.

Source Output Example:
`['acc', 'xyz', 'Bbo0Tt']`

Implementation Strategy: The Conditional Ladder



Efficiency Note:

- ▶ We iterate through the word list once, and for each word, we check the specific criteria requested.

The Full Implementation

```
def get_words_by_criteria(words, criteria=None):
    valid = ["continuous", "vowel_rich", "consonant_rich", "sorted"]
    if criteria not in valid:
        return None

    result = []
    for word in words:
        w_low = word.lower()

        if criteria == "continuous":
            if w_low.endswith("ing"):
                result.append(word)
        elif criteria == "vowel_rich":
            if sum(1 for c in w_low if c in "aeiou") > 5:
                result.append(word)
        elif criteria == "consonant_rich":
            if sum(1 for c in w_low if c.isalpha() and c not in "aeiou") > 5:
                result.append(word)
        elif criteria == "sorted":
            if word == "".join(sorted(word)):
                result.append(word)

    return result
```


Execution Trace & Verification

Input Word	Continuous	Vowel_Rich	Consonant_Rich	Sorted
running	✓ Pass	-	-	-
aeiobcdioe	-	✓ Pass	-	-
SPINNING	✓ Pass	-	✓ Pass	-
acc	-	-	-	✓ Pass
Bbo0Tt	-	-	-	✓ Pass

Visual confirmation of logic against the provided example dataset.

Complexity Analysis & Key Takeaways

Time Complexity

$O(N \times K)$

Detail: N = List Length, K = Max Word Length.

Reason: Iterating through list (N) + String operations/Sorting ($K \log K$ or K).

Space Complexity

$O(N)$

Detail: **Worst case**, returning a new list containing **all input words**.

Best Practices

- ✓ Use `..lower()` for analysis, preserve original for output.
- ✓ Generator expressions inside `sum()` save memory.
- ✓ Explicit validation for API safety (`Return None`).